

## Answers to common questions

- To run a code cell, click the play button or press **Shift + Enter**.
  - I recommend running the code blocks **one by one, without skipping any**, because later blocks may depend on the results of earlier ones.
- To search for text in the file, press **Ctrl + F** on Windows or **Command + F** on macOS.
- A script contains two types of text:
  - **Comments:** in Python, comments begin with **#**. Use everyday language to explain the purpose of the code or the logic of an algorithm. Comments are for humans; the system ignores them.
    - \* In this notebook, comments explain the code and describe tasks.
  - **Code:** instructions that must follow the programming language's syntax.
    - \* **Rule No. 1 in Python: indentation matters. Adding spaces at the beginning of a line can show that it belongs to the instruction above. Think of it as a sub-step: it tells Python, “this is part of that instruction.”**
- Ask your LLM assistant for help when you encounter:
  - A line of code you do not understand.
  - An unfamiliar concept (for example: “Explain data types in programming as if I were ten years old.”).
  - An error after running code: copy the error message and ask why it occurs and how to fix it, or share your code and ask for a correction.

Tips for exercises in this notebook:

- This notebook can be run directly in Google Colab. If you run it locally, the optional Excel and interface examples require **openpyxl** and **ipywidgets**.
- Keep the folder structure unchanged: keep **tools.py**, the **data** folder, and this notebook together in the same folder.
- Cells marked **TODO** are exercises to complete before running them.

## Basic programming concepts

- Variables
- Loops
- Conditions
- Reusing code: functions and imports

## Analogies for programming

(Illustrations in [Slides\\_programming\\_intro\\_VF.pdf](#))

1. A recipe for **pasta with Bolognese sauce**:
  - Ingredients: pasta, tomatoes, minced beef, onion, olive oil, salt, spices, etc.
  - Containers and utensils for preparing and serving the food: a pan, a plate, etc.
2. Steps:
  - Step 1: prepare the sauce:
    - Chop the tomatoes and onion.
    - Heat the pan.
    - Add olive oil, minced beef, tomatoes, spices, etc., and cook the sauce.
  - Step 2: cook the pasta (input: uncooked pasta; output: cooked pasta).
  - Step 3: mix the cooked pasta with the sauce in the pan, then serve it on a plate.
3. Interpreting the recipe using programming concepts:
  - **Variables:** we can choose different values for the ingredients.
    - Pasta: penne, fusilli, spaghetti, etc.
    - Sauce: Bolognese, pesto, etc.

- **Functions:**
  - We could buy ready-made Bolognese sauce from a supermarket: the details of step 1 and its ingredients are packaged together, much like a function hides the details of a task.
  - **ingredients** → **cooking steps** → **pasta dish** corresponds to **input** → **function** → **output**.
  - In programming, inputs are data to process: text whose font should change, a search term, an image to crop, audio to play, a file to save, etc.
- **Imports:** combine functions to do more.
  - Once a function exists, we can reuse it in another program by importing it, like picking up a ready-made tool.
  - In the physical world, duplicating a tool requires materials and time. For example, two axes require two pieces of iron and two handles. In the digital world, the additional cost of copying existing code is very small.
  - Some steps or ingredients in the recipe can be viewed as imported functions or outputs from earlier functions: cooking pasta, making pasta from flour and water, or mincing beef. The recipe already builds on earlier cooking processes.
  - A collection of functions can form software. Software can grow by adding functions or interact with other software to form an ecosystem, such as Google products or the Zotero extension for Chrome.

#### 4. Another analogy: Lego

Imagine programming as building a Lego house:

- Basic pieces: bricks for walls, a tree, or a roof.
- Assembled pieces (functions): identical trees reused to make a garden, or a section combining walls and a roof.

### ”Talk is cheap. Show me the code.” (Same concepts in real code)

Run the following cell *once* to download the course files

```
# the result should be "/content" (without double quotation marks)
# ask me if it's not the same
!pwd
```

```
/Users/jianying.liu/Workspace/Master_CEIPI/ceipi-intro-programming
```

```
# download necessary data for this notebook from this course's GitHub repository
!git clone https://github.com/JyLIU-emma/ceipi-intro-programming.git
!cp -r ceipi-intro-programming/data .
!cp ceipi-intro-programming/tools.py .
```

### Variables

```
# "print" repeats what you give it, like an echo, so you can see the value.
print("Hello, Tom!")
```

```
Hello, Tom!
```

What if the person is Julie, Elodie, Xavier, or someone else?

```
# Assign a value to x while the program is running.
x = input("Who is it? ")

print("Hello,", x, "!") # Concatenate the string with the variable.

# or you can use f-strings to format the string with the variable:
```

```
print(f"Hello, {x}!") # Insert the variable into the string as a placeholder.
```

```
Hello, Emma !  
Hello, Emma!
```

```
# Example: calculate the total price.  
starter_price = input("What is the price of the starter? ") # Ask for the first "ingredient".  
starter_price = float(starter_price) # input returns text; convert it to a number before adding.  
  
main_course_price = input("What is the price of the main course? ") # Ask for the second  
↪ "ingredient".  
main_course_price = float(main_course_price)  
  
total_price = starter_price + main_course_price  
  
print(f"The total price is {total_price} euros.")
```

```
The total price is 17.0 euros.
```

Spaces are not allowed in variable names!

```
# Optional data type example: adding two strings joins them together.  
starter_price_str = input("What is the price of the starter? ")  
main_course_price_str = input("What is the price of the main course? ")  
  
total_price_str = starter_price_str + main_course_price_str  
  
print(f"The total price is {total_price_str} euros.")
```

```
The total price is 512 euros.
```

## Functions

What if I do not want to type six lines every time I calculate the price?

```
# Define a function.  
def calculate_price():  
    starter_price = input("What is the price of the starter? ")  
    starter_price = float(starter_price)  
  
    main_course_price = input("What is the price of the main course? ")  
    main_course_price = float(main_course_price)  
  
    total_price = starter_price + main_course_price  
  
    print(f"The total price is {total_price} euros.")
```

**Important:** after writing a function definition, run the cell containing it. This makes the function available to Python so that you can call it later.

**In Python, order matters: define a function before using it.**

```
calculate_price()
```

```
The total price is 17.0 euros.
```

```
# Change the function so that it accepts inputs and returns an output.
def calculate_price_auto(starter_price, main_course_price):
    starter_price = float(starter_price)
    main_course_price = float(main_course_price)
    total_price = starter_price + main_course_price
    return total_price
```

```
calculated_total_price = calculate_price_auto(5, 12)
print(f"The total price is {calculated_total_price} euros.")
```

```
The total price is 17.0 euros.
```

### Why make this change?

- No human interaction is needed while the function runs, making automation easier.
- The function can work with other functions to perform more complex tasks.

### Importing existing functions

**Example** A file records the dish prices for ten customers, with one line per person. Each line contains two columns separated by a comma: the starter price and the main-course price, The first three lines are shown below as an example:

```
4,25
7,17
3.5,16
```

I want to find out how much the second customer should pay. How can I avoid opening the file, reading the numbers, and entering them into the function myself?

- **Main idea:**
  - Step 1: read the file and retrieve the n-th customer's prices.
  - Step 2: run our previously defined `calculate_price_auto` function.

```
# Location of the file.
file_address = "data/prices.csv"
```

```
from tools import read_price_row

client_number = 2
# Two inputs: the file path and the number of the client whose prices we want.
starter_price, main_course_price = read_price_row(file_address, client_number)

# TODO: use calculate_price_auto to calculate the total price.
# total_price =
# Answer:
total_price = calculate_price_auto(starter_price, main_course_price)

print(f"Client {client_number} must pay {total_price} euros.")
```

```
Client 2 must pay 24.0 euros.
```

Now I want to find out how much each person must pay. How can I use the file to avoid typing twenty numbers?

- **Question to consider:** are any steps redundant? Can we improve them?  
Think about code optimisation and the importance of defining each function's responsibilities.
- **Main idea:**

- step 1: Open and read the file, then **store the prices for all ten people in memory (RAM)**.
- step 2: Get the selected customer's starter and main course prices from the stored prices.
- step 3: Run the calculation.
- step 4: Repeat for all ten people.

```
from tools import read_all_prices, get_person_prices

all_prices = read_all_prices(file_address)

# TODO: use get_person_prices to retrieve the prices. Inputs: 1. all stored prices; 2. the client
↪ number

client_number = 1
# starter_price, main_course_price =
# total_price =
# Answer:
starter_price, main_course_price = get_person_prices(all_prices, client_number)
total_price = calculate_price_auto(starter_price, main_course_price)
print(f"Client {client_number} must pay {total_price} euros.")

client_number = 2
# starter_price, main_course_price =
# total_price =
# Answer:
starter_price, main_course_price = get_person_prices(all_prices, client_number)
total_price = calculate_price_auto(starter_price, main_course_price)
print(f"Client {client_number} must pay {total_price} euros.")

client_number = 3
# starter_price, main_course_price =
# total_price =
# Answer:
starter_price, main_course_price = get_person_prices(all_prices, client_number)
total_price = calculate_price_auto(starter_price, main_course_price)
print(f"Client {client_number} must pay {total_price} euros.")

# TODO: finish all...

client_number = 10
# starter_price, main_course_price =
# total_price =
# Answer:
starter_price, main_course_price = get_person_prices(all_prices, client_number)
total_price = calculate_price_auto(starter_price, main_course_price)
print(f"Client {client_number} must pay {total_price} euros.")
```

```
Client 1 must pay 29.0 euros.
Client 2 must pay 24.0 euros.
Client 3 must pay 19.5 euros.
Client 10 must pay 26.0 euros.
```

What if I do not want to repeat the function call ten times?

## Loops: for and while

```
for client_number in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:  
    # TODO: fill in the code  
    # starter_price, main_course_price =  
    # total_price =  
    # Answer:  
    starter_price, main_course_price = get_person_prices(all_prices, client_number)  
    total_price = calculate_price_auto(starter_price, main_course_price)  
    print(f"Client {client_number} must pay {total_price} euros.")
```

```
Client 1 must pay 29.0 euros.  
Client 2 must pay 24.0 euros.  
Client 3 must pay 19.5 euros.  
Client 4 must pay 14.5 euros.  
Client 5 must pay 24.5 euros.  
Client 6 must pay 9.0 euros.  
Client 7 must pay 25.0 euros.  
Client 8 must pay 27.0 euros.  
Client 9 must pay 17.0 euros.  
Client 10 must pay 26.0 euros.
```

For the while exercise, remember to advance the client number so that the loop can stop.

```
client_number = 1  
while client_number <= 10:  
    # TODO: fill in the code  
    # starter_price, main_course_price =  
    # total_price =  
    # Answer:  
    starter_price, main_course_price = get_person_prices(all_prices, client_number)  
    total_price = calculate_price_auto(starter_price, main_course_price)  
    print(f"Client {client_number} must pay {total_price} euros.")  
    client_number += 1 # important: increment the client number to avoid an infinite loop!
```

```
Client 1 must pay 29.0 euros.  
Client 2 must pay 24.0 euros.  
Client 3 must pay 19.5 euros.  
Client 4 must pay 14.5 euros.  
Client 5 must pay 24.5 euros.  
Client 6 must pay 9.0 euros.  
Client 7 must pay 25.0 euros.  
Client 8 must pay 27.0 euros.  
Client 9 must pay 17.0 euros.  
Client 10 must pay 26.0 euros.
```

## Conditions: if

**Example 1: conditional discounts** Write a function that adjusts the total price as follows:

- Students (status `student`) pay 75% of the total price.
- Clients visiting during happy hour (status `happy`) pay 85% of the total price.
- Other clients (status `normal`) pay the full price.

```
def price_after_discount(total_price):  
    client_status = input("Is the client a student (student), visiting during happy hour (happy),  
    or a regular client (normal)? ")
```

```

if client_status == "student":
    print("The client is a student and pays 75% of the total price.")
    final_price = total_price * 0.75
elif client_status == "happy":
    print("The client is visiting during happy hour and pays 85% of the total price.")
    final_price = total_price * 0.85
else:
    print("The client must pay the regular price.")
    final_price = total_price
return final_price

```

```

price_after_discount(100)
# Only "student" and "happy" trigger a discount. Any other string leads to the regular price.
# These comparisons are case-sensitive.

```

```
The client is a student and pays 75% of the total price.
```

```
75.0
```

**Example 2: Right of withdrawal (simplified version)** Under French consumer law, a consumer generally has 14 days to withdraw from a distance contract. For the purchase of goods, this period normally begins when the consumer receives the product. However, the right of withdrawal does not generally apply to products made to the consumer's specifications or clearly personalised.

For teaching purposes, this simplified simulator considers only three conditions: whether the contract was concluded at a distance, whether the product is personalised, and whether it was received no more than 14 days ago. It does not cover every legal exception and should not be treated as legal advice.

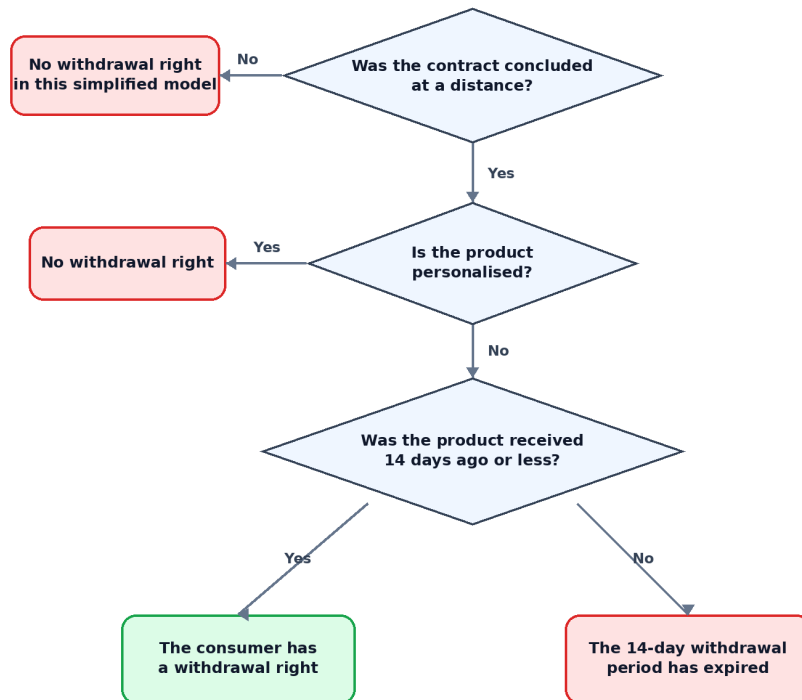
```

from IPython.display import Image, display

display(Image(filename="data/withdrawal_right_flowchart.png", height=400))

```

### Right of Withdrawal: Simplified Decision Flow



```
def check_withdrawal_right(
    is_distance_contract,
    is_personalised,
    days_since_delivery
):
    """Check whether the consumer has a withdrawal right."""
    if not is_distance_contract:
        conclusion = "No withdrawal right: the contract was not concluded at a distance."
    elif is_personalised:
        conclusion = "No withdrawal right: the product is personalised."
    elif days_since_delivery <= 14:
        conclusion = "The consumer has a withdrawal right."
    else:
        conclusion = "No withdrawal right: the 14-day period has expired."
    return conclusion
```

```
result = check_withdrawal_right(
    is_distance_contract=True,
    is_personalised=False,
    days_since_delivery=16
)
```

```
print(result)
```

```
No withdrawal right: the 14-day period has expired.
```

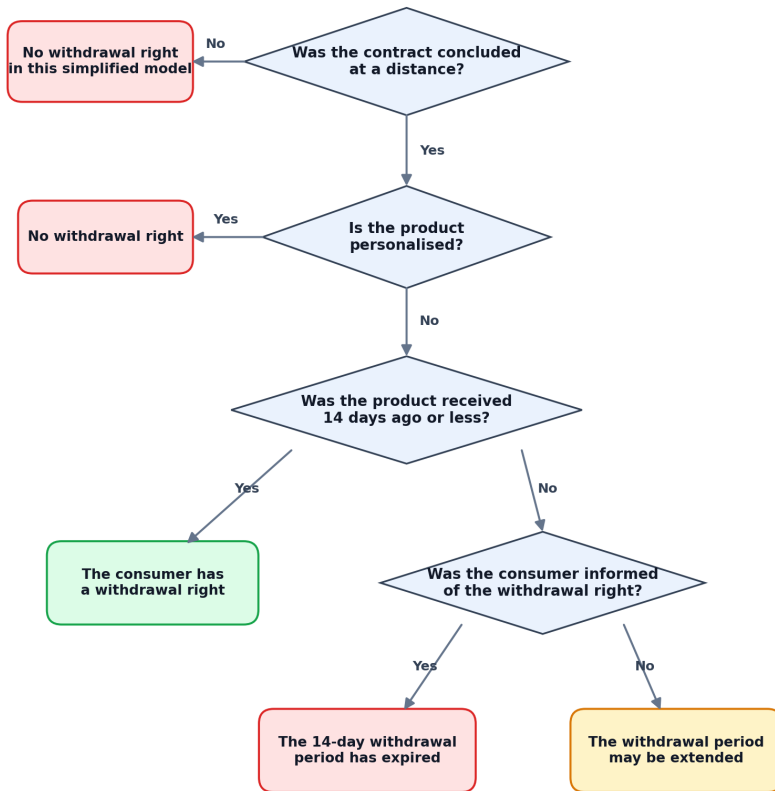
```
# Exercise: modify the function so that it can check at the end if the consumer was informed of
↳ the withdrawal right
```

*# as shown in the diagram below:*

```
from IPython.display import Image, display
```

```
display(Image(filename="data/withdrawal_right_flowchart_updated.png", height=450))
```

### Right of Withdrawal: Simplified Decision Flow



```
def check_withdrawal_right(
    is_distance_contract,
    is_personalised,
    days_since_delivery,
    was_informed
):
    # TODO: complete the function
    # Answer:
    if not is_distance_contract:
        conclusion = "No withdrawal right: the contract was not concluded at a distance."
    elif is_personalised:
        conclusion = "No withdrawal right: the product is personalised."
    elif days_since_delivery <= 14:
        conclusion = "The consumer has a withdrawal right."
    elif not was_informed:
        conclusion = "The withdrawal period may be extended since the consumer was not informed
↳ of their right to withdraw."
    else:
        conclusion = "No withdrawal right: the 14-day period has expired and the consumer was
↳ informed of their right to withdraw."
```

```
return conclusion
```

```
result = check_withdrawal_right(  
    is_distance_contract=True,  
    is_personalised=False,  
    days_since_delivery=16,  
    was_informed=False  
)  
print(result)
```

The withdrawal period may be extended since the consumer was not informed of their right to  
↪ withdraw.

```
from tools import read_excel_lists, get_client_prices
```

```
# Examples of how to use these two functions.
```

```
excel_path = "data/orders_menu.xlsx"  
orders, menu = read_excel_lists(excel_path)
```

```
client_number = 1  
starter_price, main_course_price = get_client_prices(client_number, orders, menu)  
starter_price, main_course_price
```

Optional: use existing tools to build more complex things

```
(8.0, 14.0)
```

menu

```
[('Onion soup', 'Starter', 6.5),  
 ('Warm goat cheese salad', 'Starter', 8.0),  
 ('Eggs with mayonnaise', 'Starter', 5.5),  
 ('Bacon and egg tart', 'Starter', 7.5),  
 ('Country-style terrine', 'Starter', 8.5),  
 ('Snails in parsley butter', 'Starter', 10.0),  
 ('Burgundy beef stew', 'Main course', 18.5),  
 ('Herb-roasted chicken', 'Main course', 16.0),  
 ('Grilled salmon', 'Main course', 19.5),  
 ('Provençal vegetable stew', 'Main course', 14.0),  
 ('Slow-cooked duck', 'Main course', 21.0),  
 ('Steak and chips', 'Main course', 17.5)]
```

orders

```
[(3.0, 'Country-style terrine', 'Burgundy beef stew'),  
 (2.0, 'Snails in parsley butter', 'Grilled salmon'),  
 (8.0, 'Onion soup', 'Burgundy beef stew'),  
 (6.0, 'Bacon and egg tart', 'Provençal vegetable stew'),  
 (4.0, 'Onion soup', 'Slow-cooked duck'),  
 (1.0, 'Warm goat cheese salad', 'Provençal vegetable stew'),  
 (5.0, 'Country-style terrine', 'Herb-roasted chicken'),
```

```
(10.0, 'Eggs with mayonnaise', 'Slow-cooked duck'),
(9.0, 'Bacon and egg tart', 'Steak and chips'),
(7.0, 'Eggs with mayonnaise', 'Slow-cooked duck')]
```

*# TODO: complete the calculate\_client\_total function.*

*# Answer: see tools.py.*

```
def calculate_client_total(client_number, excel_path):
    # Read the information from the Excel file.
    # orders, menu =
    # Answer:
    orders, menu = read_excel_lists(excel_path)

    # Get the prices for the client identified by client_number.
    # starter_price, main_course_price =
    # Answer:
    starter_price, main_course_price = get_client_prices(client_number, orders, menu)

    # Calculate the total price using calculate_price_auto.
    # total_price =
    # Answer:
    total_price = calculate_price_auto(starter_price, main_course_price)
    return total_price

calculate_client_total(1, excel_path)
```

22.0

```
from tools import show_menu_interface

show_menu_interface(excel_path)
```

Programmers like open source, so of course you can download and import functions/packages created by other programmers.

- To share any kind of code: use GitHub
- To share Python Package: publish them on **PyPI**(the Python Package Index), download with terminal command `pip install [package-name]`

*# to run a terminal command in a Jupyter Notebook, you can use the "!" prefix. For example, to  
 ↪ install a package, you can run:*

```
!pip install dateparser
```

```
Collecting dateparser
  Downloading dateparser-1.4.3-py3-none-any.whl.metadata (25 kB)
Requirement already satisfied: python-dateutil>=2.7.0 in
  ↪ /opt/miniconda3/envs/creativity_analysis_env/lib/python3.11/site-packages (from dateparser)
  ↪ (2.9.0.post0)
Requirement already satisfied: pytz>=2024.2 in
  ↪ /opt/miniconda3/envs/creativity_analysis_env/lib/python3.11/site-packages (from dateparser)
  ↪ (2025.2)
Requirement already satisfied: regex>=2024.9.11 in
  ↪ /opt/miniconda3/envs/creativity_analysis_env/lib/python3.11/site-packages (from dateparser)
  ↪ (2025.11.3)
Collecting tzlocal>=0.2 (from dateparser)
```

```

Downloading tzlocal-5.4.4-py3-none-any.whl.metadata (7.7 kB)
Requirement already satisfied: six>=1.5 in
  ↪ /opt/miniconda3/envs/creativity_analysis_env/lib/python3.11/site-packages (from
  ↪ python-dateutil>=2.7.0->dateparser) (1.17.0)
Downloading dateparser-1.4.3-py3-none-any.whl (322 kB)
Downloading tzlocal-5.4.4-py3-none-any.whl (18 kB)
Installing collected packages: tzlocal, dateparser
===== 2/2 [dateparser]2 [dateparser]
Successfully installed dateparser-1.4.3 tzlocal-5.4.4

```

*# then you can import and use the package in your code:*

```

import dateparser

raw_dates = ["Wednesday, 1st October", "October 1872", "1872-10-09"]

for d in raw_dates:
    parsed = dateparser.parse(d, settings={"PREFER_DAY_OF_MONTH": "first"})
    print(f'"{d}" -> {parsed.strftime("%Y-%m-%d")} if parsed else "Failed"')

```

```

'Wednesday, 1st October' -> 2026-10-01
'October 1872' -> 1872-10-01
'1872-10-09' -> 1872-10-09

```

## Programming in the age of LLMs: from implementation to planning and validation

- What makes a good prompt? Express your needs clearly and provide the right level of detail.
- Guide the coding agent to follow your design and produce code that is easy to maintain: minimizing coupling
- Be aware of hallucinations: human evaluation matters.
- human-in-the-loop: validate LLM-generated results, as they may not meet your requirements.
- Vibe coding project: the environmental impact of using LLMs.

### Using LLMs to help you program

1. The following code cell has 3 problems. Use an LLM to help you debug the code:

```

def calculate_total_to_pay(identity, client_number, excel_path):
    """Calculate the total price to pay for a client identified by their identity."""
    orders, menu = read_excel_lists(excel_path)
    starter_price, main_course_price = get_client_prices(client_number, orders, menu)
    total_price = calculate_price_auto(starter_price, main_course_price)
    if identity == "student":
        total_price = total_price * 0.75
    elif identity == "happy":
        total_price = total_price * 0.85
    else
        total_price = total_price
    return total_price

# excel_path = "data/orders_menu.xlsx"
excel_path = "data/order_menu.xlsx"

```

```
customer_num = 1
final_price = calculate_total_to_pay("student", customer_num, excel_path)
print(f"Customer {customer_num} must pay {final_price} euros.")
```

```
Cell In[32], line 6
    if identity = "student":
        ^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

1. Run the cell to test the code. What do you see?
2. Now, how can we correct this error ? We can ask an LLM to fix it.

You can use the following prompt examples:

- Prompt example 1:

I encountered an error message:  
 [error message copied here]  
 can you help me fix it?

- Prompt example 2:

I encountered an error when trying to run this snippet of code.  
 [code copied here]  
 The error message is as follows:  
 [error message copied here]  
 Can you help me fix it? Please use the following format:

1. Identify where the error occurs, for example, the line number.
2. Briefly explain the problem as if you were explaining it to someone who has just started learning to program.
3. Suggest a solution.

Example response I got: <https://share.gemini.google/5vhdv0RKrMlc>

Which answer meets your needs better? Which prompt do you think is better?

## 2. Use an LLM as your pair programmer

Exercise: anonymising a document (Example conversation: <https://share.gemini.google/pZAMzT8umx7B>)

Write a prompt asking an LLM to create a function that masks dates in text. For example:

- input: "On 10/02/1872, Phileas Fogg accepted a bet."
- output: "On [DATE], Phileas Fogg accepted a bet."

```
import re

def mask_dates(text: str) -> str:
    # Matches patterns like DD/MM/YYYY, MM-DD-YYYY, YYYY-MM-DD, etc.
    date_pattern = r"\b(?:\d{1,4}[-/\.]\d{1,2}[-/\.]\d{1,4})\b"
    return re.sub(date_pattern, "[DATE]", text)

# Example usage:
sentence = "On 10/02/1872, Phileas Fogg accepted a bet."
print(mask_dates(sentence))
```

```
# Output: "On [DATE], Phileas Fogg accepted a bet."
```

```
On [DATE], Phileas Fogg accepted a bet.
```

Then, you encounter dates containing only a month and a day, or only a year, as well as dates written like this:

- "Phileas Fogg, having won twenty guineas at whist, and taken leave of his friends, left the Reform Club at ten minutes to eight. Passepartout had conscientiously studied his program; and it was noted down that on Wednesday, 2nd October, they were to start from London at 8:45 P.M."

Work with an LLM to develop a new function that can handle all these cases.

Possible approaches (you can include these technical details in your prompt):

- rule-based (something close to expert systems or old-fashioned AI): more sophisticated regex rules
- Pre-Trained NER(=Named Entity Recognition) models via spaCy (something closer to modern AI, machine learning)
  - You need to download the package and pretrained model before using it

```
# code written by Gemini
```

```
# 1. use regex to find dates in various formats
```

```
# TODO:
```

```
# one possible Answer:
```

```
import re
```

```
MONTHS = (
```

```
    r"(:Jan(?:uary)?|Feb(?:ruary)?|Mar(?:ch)?|Apr(?:il)?|May|Jun(?:e)?|"
```

```
    r"Jul(?:y)?|Aug(?:ust)?|Sep(?:tember)?|Oct(?:ober)?|Nov(?:ember)?|Dec(?:ember)?)"
```

```
)
```

```
DAYS_OF_WEEK = r"(:Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday)"
```

```
PATTERNS = [
```

```
    # 1. Date including the day of the week: "Wednesday, 2nd October 1872" or "Wednesday, 2nd  
    ↪ October"
```

```
    re.compile(  
        rf"\b{DAYS_OF_WEEK},\s+\d{{1,2}}(?:st|nd|rd|th)?\s+{MONTHS}(?:\s+\d{{4}})?\b",  
        re.IGNORECASE,  
    ),
```

```
    # 2. Standard ISO / Numeric: "1872-10-02", "10/02/1872", "02.10.1872"
```

```
    re.compile(  
        rf"\b(?:\d{{4}}[-/.]\d{{1,2}}[-/.]\d{{1,2}}|\d{{1,2}}[-/.]\d{{1,2}}[-/.]\d{{2,4}})\b"
```

```
    ),
```

```
    # 3. Month & Day (both orders): "2nd October", "October 2nd", "Oct 2, 1872"
```

```
    re.compile(  
        rf"\b(?:\d{{1,2}}(?:st|nd|rd|th)?\s+(?:of\s+)?{MONTHS}|{MONTHS}\s+\d{{1,2}}(?:st|nd|rd|  
        ↪ th)?(?:[,,\s]+\d{{4}})?\b",  
        re.IGNORECASE,  
    ),
```

```
    # 4. Standalone Month + Year: "October 1872"
```

```
    re.compile(rf"\b{MONTHS}\s+\d{{4}}\b", re.IGNORECASE),
```

```
    # 5. Standalone Year with contextual preposition: "in 1872", "during 1905"
```

```
    re.compile(r"\b(?:in|during|by|around|year)\s+(\d{{4}})\b", re.IGNORECASE),
```

```
]
```

```
def mask_dates_regex(text: str, mask_token: str = "[DATE]") -> str:
    spans = []

    # 1. get date spans
    for pattern in PATTERNS:
        for match in pattern.finditer(text):
            start, end = match.span()
            # Ensure this match doesn't overlap with an already captured longer date
            if not any(
                s_start <= start and end <= s_end for s_start, s_end in spans
            ):
                spans.append((start, end))

    # 2. sort spans by their position in the text, from back to front
    spans.sort(key=lambda x: x[0], reverse=True)

    # 3. replace spans with mask token
    masked_text = text
    for start, end in spans:
        masked_text = masked_text[:start] + mask_token + masked_text[end:]

    return masked_text
```

```
# You may need to install spaCy and download the English model first:
# !pip install spacy
# !python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0|
  ↪ /en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
  ===== 12.8/12.8 MB 31.2 MB/s eta 0:00:00a 0:00:01
Installing collected packages: en-core-web-sm
Successfully installed en-core-web-sm-3.8.0
[ok] Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
```

```
# 2. use pre-trained models via spaCy

# One possible answer:
import spacy

# Load the English model outside of the function to avoid reloading it every time the function is
↪ called.
nlp = spacy.load("en_core_web_sm")

def mask_dates_spacy(
    text: str, mask_token: str = "[DATE]", model=nlp
) -> str:
    """Use spaCy to locate all entities labeled as DATE and replace them with the mask token."""
    doc = model(text)

    # 1. Filter out all entities labeled as DATE
    date_ents = [ent for ent in doc.ents if ent.label_ == "DATE"]
```

```

# 2. Sort by character start position from back to front (reverse order replacement to
↳ prevent index offset)
date_ents.sort(key=lambda ent: ent.start_char, reverse=True)

# 3. Replace spans with mask token
masked_text = text
for ent in date_ents:
    masked_text = (
        masked_text[: ent.start_char] + mask_token + masked_text[ent.end_char :]
    )

return masked_text

```

```

/opt/miniconda3/envs/creativity_analysis_env/lib/python3.11/site-packages/tqdm/auto.py:21:
↳ TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
↳ https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

```

```

On [DATE], Phileas Fogg accepted the bet. He arrived in Bombay on [DATE] and completed the
↳ voyage in [DATE].

```

```

# --- Example Usage ---
sent = ("Phileas Fogg, having won twenty guineas at whist, and taken leave of his friends, left
↳ the Reform Club at ten minutes to eight. Passepartout had conscientiously studied his
↳ program; and it was noted down that on Wednesday, 2nd October, they were to start from London
↳ at 8:45 P.M.\n"
        "On Wednesday, 2nd October, Phileas Fogg accepted the bet.\n"
        "He arrived in Bombay on 20 October and completed the voyage in 1872.\n")
sent_masked_regex = mask_dates_regex(sent)
print(sent_masked_regex)

sent_masked_spacy = mask_dates_spacy(sent)
print(sent_masked_spacy)

```

```

Phileas Fogg, having won twenty guineas at whist, and taken leave of his friends, left the
↳ Reform Club at ten minutes to eight. Passepartout had conscientiously studied his program;
↳ and it was noted down that on [DATE], they were to start from London at 8:45 P.M.
On [DATE], Phileas Fogg accepted the bet.
He arrived in Bombay on [DATE] and completed the voyage [DATE].

```

```

Phileas Fogg, having won twenty guineas at whist, and taken leave of his friends, left the
↳ Reform Club at ten minutes to eight. Passepartout had conscientiously studied his program;
↳ and it was noted down that on [DATE], they were to start from London at 8:45 P.M.
On [DATE], Phileas Fogg accepted the bet.
He arrived in Bombay on [DATE] and completed the voyage in [DATE].

```

### 3. Validating LLM-generated code

- You can review the code
- Or design tests to check its results

Based on the results of code review and testing, continue improving the code or redefine the task you give to the LLM.

Prompt used for creating this validation test:

I want to create a test dataset and supporting code to evaluate a function that masks dates in  
↪ text. The requirements are:

1. Create 100 English sentences, each containing 1-2 date expressions. Use a wide variety of  
↪ formats, including ISO dates, standalone years, month-and-day combinations, and dates  
↪ containing days of the week. Include both full names and abbreviations.
2. Save the dataset as a CSV file with three columns: ``text``, ``masked_text``, and ``date_span``. For  
↪ example:
  - ``text``: On Wednesday, 2nd October, Phileas Fogg accepted the bet.
  - ``masked_text``: On [DATE], Phileas Fogg accepted the bet.
  - ``date_span``: Wednesday, 2nd October
3. The function to test, ``mask_dates_func``, already exists. It takes the original sentence from  
↪ ``text`` as input and should return the expected result in ``masked_text``.

Write code that tests the function on every sentence by comparing its output with the expected  
↪ ``masked_text``.

- If all tests pass, print ``"function passed"`` at the end.
- Otherwise, print the accuracy at the end.
- For each failed test, print the original sentence, the expected output, the actual output, and  
↪ the date text that should have been identified.

```
import csv

def check_mask_dates(mask_dates_func, csv_path="data/date_masking_test.csv"):
    """Check a masking function against the expected output in each CSV row."""
    correct = 0
    total = 0

    with open(csv_path, encoding="utf-8-sig", newline="") as file:
        for row in csv.DictReader(file):
            total += 1
            try:
                actual = mask_dates_func(row["text"])
                passed = actual == row["masked_text"]
            except Exception as error:
                actual = f"{type(error).__name__}: {error}"
                passed = False

            if passed:
                correct += 1
            else:
                print(f"\nFailed case {total}")
                print("Original:", row["text"])
                print("Expected:", row["masked_text"])
                print("Actual:", repr(actual))
                print("Expected date spans:", row["date_span"])

    if total == 0:
        print("No test cases found.")
    elif correct == total:
        print("function passed")
```

```
else:  
    print(f"Accuracy: {correct / total:.2%} ({correct}/{total})")
```

```
check_mask_dates(mask_dates_regex)
```

```
Failed case 3  
Original: Please submit the form by 2025-12-31.  
Expected: Please submit the form by [DATE].  
Actual: 'Please submit the form [DATE]E.'  
Expected date spans: 2025-12-31  
  
Failed case 19  
Original: The museum opened in 1987.  
Expected: The museum opened in [DATE].  
Actual: 'The museum opened [DATE].'  
Expected date spans: 1987  
  
Failed case 20  
Original: She graduated in 2001.  
Expected: She graduated in [DATE].  
Actual: 'She graduated [DATE].'  
Expected date spans: 2001  
  
Failed case 21  
Original: The company was founded in 1998.  
Expected: The company was founded in [DATE].  
Actual: 'The company was founded [DATE].'  
Expected date spans: 1998  
  
Failed case 22  
Original: The report covers developments in 2024.  
Expected: The report covers developments in [DATE].  
Actual: 'The report covers developments [DATE].'  
Expected date spans: 2024  
  
Failed case 23  
Original: The painting was restored in 1875.  
Expected: The painting was restored in [DATE].  
Actual: 'The painting was restored [DATE].'  
Expected date spans: 1875  
  
Failed case 24  
Original: The new policy takes effect in 2030.  
Expected: The new policy takes effect in [DATE].  
Actual: 'The new policy takes effect [DATE].'  
Expected date spans: 2030  
  
Failed case 50  
Original: The document is dated Feb. 14, 2025.  
Expected: The document is dated [DATE].  
Actual: 'The document is dated Feb. 14, 2025.'  
Expected date spans: Feb. 14, 2025
```

Failed case 52

Original: The next review is on 7 Apr. 2024.

Expected: The next review is on [DATE].

Actual: 'The next review is on [DATE]. 2024.'

Expected date spans: 7 Apr. 2024

Failed case 54

Original: The delivery is expected on 9 Jul. 2026.

Expected: The delivery is expected on [DATE].

Actual: 'The delivery is expected on [DATE]. 2026.'

Expected date spans: 9 Jul. 2026

Failed case 55

Original: The form was completed on Aug. 10, 2024.

Expected: The form was completed on [DATE].

Actual: 'The form was completed on Aug. 10, 2024.'

Expected date spans: Aug. 10, 2024

Failed case 57

Original: The notice expires on 12 Sept. 2026.

Expected: The notice expires on [DATE].

Actual: 'The notice expires on 12 Sept. 2026.'

Expected date spans: 12 Sept. 2026

Failed case 59

Original: The reply arrived on 14 Nov. 2025.

Expected: The reply arrived on [DATE].

Actual: 'The reply arrived on [DATE]. 2025.'

Expected date spans: 14 Nov. 2025

Failed case 60

Original: The tickets are valid for Dec. 15, 2026.

Expected: The tickets are valid for [DATE].

Actual: 'The tickets are valid for Dec. 15, 2026.'

Expected date spans: Dec. 15, 2026

Failed case 67

Original: The invitations mention Jul. 7 and Jul. 14.

Expected: The invitations mention [DATE] and [DATE].

Actual: 'The invitations mention Jul. 7 and Jul. 14.'

Expected date spans: ["Jul. 7", "Jul. 14"]

Failed case 69

Original: The office closes on Sept. 9 and reopens on Sept. 16.

Expected: The office closes on [DATE] and reopens on [DATE].

Actual: 'The office closes on Sept. 9 and reopens on Sept. 16.'

Expected date spans: ["Sept. 9", "Sept. 16"]

Failed case 70

Original: We will meet on 10 Oct. and review the results on 17 Oct..

Expected: We will meet on [DATE] and review the results on [DATE].

Actual: 'We will meet on [DATE]. and review the results on [DATE]..'

Expected date spans: ["10 Oct.", "17 Oct."]

Failed case 73

Original: The workshop starts on Monday and ends on Tuesday.  
Expected: The workshop starts on [DATE] and ends on [DATE].  
Actual: 'The workshop starts on Monday and ends on Tuesday.'  
Expected date spans: ["Monday", "Tuesday"]

Failed case 74

Original: The witness is available on Wednesday or Thursday.  
Expected: The witness is available on [DATE] or [DATE].  
Actual: 'The witness is available on Wednesday or Thursday.'  
Expected date spans: ["Wednesday", "Thursday"]

Failed case 75

Original: The cafe opens on Friday and closes on Sunday.  
Expected: The cafe opens on [DATE] and closes on [DATE].  
Actual: 'The cafe opens on Friday and closes on Sunday.'  
Expected date spans: ["Friday", "Sunday"]

Failed case 76

Original: The match moved from Saturday to Monday.  
Expected: The match moved from [DATE] to [DATE].  
Actual: 'The match moved from Saturday to Monday.'  
Expected date spans: ["Saturday", "Monday"]

Failed case 77

Original: The timetable lists Mon and Wed as teaching days.  
Expected: The timetable lists [DATE] and [DATE] as teaching days.  
Actual: 'The timetable lists Mon and Wed as teaching days.'  
Expected date spans: ["Mon", "Wed"]

Failed case 78

Original: The office accepts visitors on Tue. and Thu..  
Expected: The office accepts visitors on [DATE] and [DATE].  
Actual: 'The office accepts visitors on Tue. and Thu..  
Expected date spans: ["Tue.", "Thu."]

Failed case 79

Original: The shifts fall on Tues and Thurs.  
Expected: The shifts fall on [DATE] and [DATE].  
Actual: 'The shifts fall on Tues and Thurs.'  
Expected date spans: ["Tues", "Thurs"]

Failed case 80

Original: The rehearsals are on Fri. and Sat..  
Expected: The rehearsals are on [DATE] and [DATE].  
Actual: 'The rehearsals are on Fri. and Sat..  
Expected date spans: ["Fri.", "Sat."]

Failed case 81

Original: The notice lists Sun and Mon. as closure days.  
Expected: The notice lists [DATE] and [DATE] as closure days.  
Actual: 'The notice lists Sun and Mon. as closure days.'  
Expected date spans: ["Sun", "Mon."]

Failed case 82

Original: The meeting moved from Monday, January 15, 2024 to Tuesday, January 16, 2024.

Expected: The meeting moved from [DATE] to [DATE].  
Actual: 'The meeting moved from Monday, [DATE] to Tuesday, [DATE].'  
Expected date spans: ["Monday, January 15, 2024", "Tuesday, January 16, 2024"]

Failed case 83

Original: The hearing began on Wed, 2 Oct 2024 and ended on Thu, 3 Oct 2024.  
Expected: The hearing began on [DATE] and ended on [DATE].  
Actual: 'The hearing began on Wed, [DATE] and ended on Thu, [DATE].'  
Expected date spans: ["Wed, 2 Oct 2024", "Thu, 3 Oct 2024"]

Failed case 84

Original: The workshop runs from Fri. 4 October 2024 to Sat. 5 October 2024.  
Expected: The workshop runs from [DATE] to [DATE].  
Actual: 'The workshop runs from Fri. [DATE] to Sat. [DATE].'  
Expected date spans: ["Fri. 4 October 2024", "Sat. 5 October 2024"]

Failed case 85

Original: The gallery is open on Sunday 6th October and Monday 7th October.  
Expected: The gallery is open on [DATE] and [DATE].  
Actual: 'The gallery is open on Sunday [DATE] and Monday [DATE].'  
Expected date spans: ["Sunday 6th October", "Monday 7th October"]

Failed case 86

Original: The first session is on Tue., Oct. 8, 2024 and the second is on Wed., Oct. 9, 2024.  
Expected: The first session is on [DATE] and the second is on [DATE].  
Actual: 'The first session is on Tue., Oct. 8, 2024 and the second is on Wed., Oct. 9, 2024.'  
Expected date spans: ["Tue., Oct. 8, 2024", "Wed., Oct. 9, 2024"]

Failed case 88

Original: The organization began in 1995 and expanded in 2005.  
Expected: The organization began in [DATE] and expanded in [DATE].  
Actual: 'The organization began [DATE] and expanded [DATE].'  
Expected date spans: ["1995", "2005"]

Failed case 90

Original: The notice was sent on Feb. 28, 2024 and received on 2024-02-29.  
Expected: The notice was sent on [DATE] and received on [DATE].  
Actual: 'The notice was sent on Feb. 28, 2024 and received on [DATE].'  
Expected date spans: ["Feb. 28, 2024", "2024-02-29"]

Failed case 94

Original: The archive contains letters from 1888 and 1902.  
Expected: The archive contains letters from [DATE] and [DATE].  
Actual: 'The archive contains letters from 1888 and 1902.'  
Expected date spans: ["1888", "1902"]

Failed case 99

Original: The 2024-page manuscript was submitted in 2025 and published in 2026.  
Expected: The 2024-page manuscript was submitted in [DATE] and published in [DATE].  
Actual: 'The 2024-page manuscript was submitted [DATE] and published [DATE].'  
Expected date spans: ["2025", "2026"]  
Accuracy: 65.00% (65/100)

You can send the test results and your code to an LLM and ask it to improve the code. The following snippet shows an improved version.

```

# You only need to import the same module once in a Python script.
# import re

MONTHS = (
    r"(?:Jan(?:uary)?|Feb(?:ruary)?|Mar(?:ch)?|Apr(?:il)?|May|Jun(?:e)?|"
    r"Jul(?:y)?|Aug(?:ust)?|Sep(?:tember)?|Oct(?:ober)?|Nov(?:ember)?|Dec(?:ember)?)"
)

DAYS_OF_WEEK = (
    r"(?:Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday)"
)

PATTERNS = [
    # 1. Date with weekday: "Wednesday, 2nd October 1872"
    re.compile(
        rf"\b{DAYS_OF_WEEK},\s+\d{{1,2}}(?:st|nd|rd|th)?"
        rf"\s+{MONTHS}(?:\s+\d{{4}})?\b",
        re.IGNORECASE,
    ),

    # 2. Numeric date: "1872-10-02", "10/02/1872", "02.10.1872"
    re.compile(
        rf"\b(?:\d{{4}}[-./]\d{{1,2}}[-./]\d{{1,2}}|"
        rf"\d{{1,2}}[-./]\d{{1,2}}[-./]\d{{2,4}})\b"
    ),

    # 3. Month and day: "2nd October", "October 2nd", "Oct 2, 1872"
    re.compile(
        rf"\b(?:"
        rf"\d{{1,2}}(?:st|nd|rd|th)?\s+(?:of\s+)?{MONTHS}|"
        rf"{MONTHS}\s+\d{{1,2}}(?:st|nd|rd|th)?"
        rf")\s+[\s]+\d{{4}})?\b",
        re.IGNORECASE,
    ),

    # 4. Month and year: "October 1872"
    re.compile(
        rf"\b{MONTHS}\s+\d{{4}}\b",
        re.IGNORECASE,
    ),

    # 5. Year with context: capture the year without the preceding word.
    re.compile(
        rf"\b(?:in|during|by|around|year)\s+(?P<date>\d{{4}})\b",
        re.IGNORECASE,
    ),
]

def mask_dates_regex_new(text: str, mask_token: str = "[DATE]") -> str:
    spans = []

    # Collect date positions in the original text.
    for pattern in PATTERNS:
        for match in pattern.finditer(text):

```

```

    # Use the named date group when the pattern provides one.
    if "date" in pattern.groupindex:
        start, end = match.span("date")
    else:
        start, end = match.span()

    # Skip matches that overlap with an accepted date.
    if not any(
        start < saved_end and end > saved_start
        for saved_start, saved_end in spans
    ):
        spans.append((start, end))

    # Replace from right to left to preserve earlier positions.
    spans.sort(reverse=True)

    for start, end in spans:
        text = text[:start] + mask_token + text[end:]

    return text

```

```

# check again:
check_mask_dates(mask_dates_regex_new)

```

```

Failed case 50
Original: The document is dated Feb. 14, 2025.
Expected: The document is dated [DATE].
Actual: 'The document is dated Feb. 14, 2025.'
Expected date spans: Feb. 14, 2025

Failed case 52
Original: The next review is on 7 Apr. 2024.
Expected: The next review is on [DATE].
Actual: 'The next review is on [DATE]. 2024.'
Expected date spans: 7 Apr. 2024

Failed case 54
Original: The delivery is expected on 9 Jul. 2026.
Expected: The delivery is expected on [DATE].
Actual: 'The delivery is expected on [DATE]. 2026.'
Expected date spans: 9 Jul. 2026

Failed case 55
Original: The form was completed on Aug. 10, 2024.
Expected: The form was completed on [DATE].
Actual: 'The form was completed on Aug. 10, 2024.'
Expected date spans: Aug. 10, 2024

Failed case 57
Original: The notice expires on 12 Sept. 2026.
Expected: The notice expires on [DATE].
Actual: 'The notice expires on 12 Sept. 2026.'

```

Expected date spans: 12 Sept. 2026

Failed case 59

Original: The reply arrived on 14 Nov. 2025.

Expected: The reply arrived on [DATE].

Actual: 'The reply arrived on [DATE]. 2025.'

Expected date spans: 14 Nov. 2025

Failed case 60

Original: The tickets are valid for Dec. 15, 2026.

Expected: The tickets are valid for [DATE].

Actual: 'The tickets are valid for Dec. 15, 2026.'

Expected date spans: Dec. 15, 2026

Failed case 67

Original: The invitations mention Jul. 7 and Jul. 14.

Expected: The invitations mention [DATE] and [DATE].

Actual: 'The invitations mention Jul. 7 and Jul. 14.'

Expected date spans: ["Jul. 7", "Jul. 14"]

Failed case 69

Original: The office closes on Sept. 9 and reopens on Sept. 16.

Expected: The office closes on [DATE] and reopens on [DATE].

Actual: 'The office closes on Sept. 9 and reopens on Sept. 16.'

Expected date spans: ["Sept. 9", "Sept. 16"]

Failed case 70

Original: We will meet on 10 Oct. and review the results on 17 Oct..

Expected: We will meet on [DATE] and review the results on [DATE].

Actual: 'We will meet on [DATE]. and review the results on [DATE]..'

Expected date spans: ["10 Oct.", "17 Oct."]

Failed case 73

Original: The workshop starts on Monday and ends on Tuesday.

Expected: The workshop starts on [DATE] and ends on [DATE].

Actual: 'The workshop starts on Monday and ends on Tuesday.'

Expected date spans: ["Monday", "Tuesday"]

Failed case 74

Original: The witness is available on Wednesday or Thursday.

Expected: The witness is available on [DATE] or [DATE].

Actual: 'The witness is available on Wednesday or Thursday.'

Expected date spans: ["Wednesday", "Thursday"]

Failed case 75

Original: The cafe opens on Friday and closes on Sunday.

Expected: The cafe opens on [DATE] and closes on [DATE].

Actual: 'The cafe opens on Friday and closes on Sunday.'

Expected date spans: ["Friday", "Sunday"]

Failed case 76

Original: The match moved from Saturday to Monday.

Expected: The match moved from [DATE] to [DATE].

Actual: 'The match moved from Saturday to Monday.'

Expected date spans: ["Saturday", "Monday"]

Failed case 77

Original: The timetable lists Mon and Wed as teaching days.  
Expected: The timetable lists [DATE] and [DATE] as teaching days.  
Actual: 'The timetable lists Mon and Wed as teaching days.'  
Expected date spans: ["Mon", "Wed"]

Failed case 78

Original: The office accepts visitors on Tue. and Thu..  
Expected: The office accepts visitors on [DATE] and [DATE].  
Actual: 'The office accepts visitors on Tue. and Thu..'  
Expected date spans: ["Tue.", "Thu."]

Failed case 79

Original: The shifts fall on Tues and Thurs.  
Expected: The shifts fall on [DATE] and [DATE].  
Actual: 'The shifts fall on Tues and Thurs.'  
Expected date spans: ["Tues", "Thurs"]

Failed case 80

Original: The rehearsals are on Fri. and Sat..  
Expected: The rehearsals are on [DATE] and [DATE].  
Actual: 'The rehearsals are on Fri. and Sat..'  
Expected date spans: ["Fri.", "Sat."]

Failed case 81

Original: The notice lists Sun and Mon. as closure days.  
Expected: The notice lists [DATE] and [DATE] as closure days.  
Actual: 'The notice lists Sun and Mon. as closure days.'  
Expected date spans: ["Sun", "Mon."]

Failed case 82

Original: The meeting moved from Monday, January 15, 2024 to Tuesday, January 16, 2024.  
Expected: The meeting moved from [DATE] to [DATE].  
Actual: 'The meeting moved from Monday, [DATE] to Tuesday, [DATE].'  
Expected date spans: ["Monday, January 15, 2024", "Tuesday, January 16, 2024"]

Failed case 83

Original: The hearing began on Wed, 2 Oct 2024 and ended on Thu, 3 Oct 2024.  
Expected: The hearing began on [DATE] and ended on [DATE].  
Actual: 'The hearing began on Wed, [DATE] and ended on Thu, [DATE].'  
Expected date spans: ["Wed, 2 Oct 2024", "Thu, 3 Oct 2024"]

Failed case 84

Original: The workshop runs from Fri. 4 October 2024 to Sat. 5 October 2024.  
Expected: The workshop runs from [DATE] to [DATE].  
Actual: 'The workshop runs from Fri. [DATE] to Sat. [DATE].'  
Expected date spans: ["Fri. 4 October 2024", "Sat. 5 October 2024"]

Failed case 85

Original: The gallery is open on Sunday 6th October and Monday 7th October.  
Expected: The gallery is open on [DATE] and [DATE].  
Actual: 'The gallery is open on Sunday [DATE] and Monday [DATE].'  
Expected date spans: ["Sunday 6th October", "Monday 7th October"]

Failed case 86

Original: The first session is on Tue., Oct. 8, 2024 and the second is on Wed., Oct. 9, 2024.

Expected: The first session is on [DATE] and the second is on [DATE].

Actual: 'The first session is on Tue., Oct. 8, 2024 and the second is on Wed., Oct. 9, 2024.'

Expected date spans: ["Tue., Oct. 8, 2024", "Wed., Oct. 9, 2024"]

Failed case 90

Original: The notice was sent on Feb. 28, 2024 and received on 2024-02-29.

Expected: The notice was sent on [DATE] and received on [DATE].

Actual: 'The notice was sent on Feb. 28, 2024 and received on [DATE].'

Expected date spans: ["Feb. 28, 2024", "2024-02-29"]

Failed case 94

Original: The archive contains letters from 1888 and 1902.

Expected: The archive contains letters from [DATE] and [DATE].

Actual: 'The archive contains letters from 1888 and 1902.'

Expected date spans: ["1888", "1902"]

Accuracy: 74.00% (74/100)

**TODO:** Keep improving the function. Can you achieve 100% accuracy on this test set ?

(But remember: passing every test in this set does not guarantee that the function will work correctly on new examples.)

This whole process:

A human defines the task --> AI writes the code --> the human tests the results and redefines the requirements --> AI revises the code --> the human checks again...

is called **human-in-the-loop** workflow.

**Don't blindly trust AI-generated code.**

**Stay involved, test the results,  
and keep control over the changes.**

## A Project from scratch: the environmental impact of AI

Ref: Anne-Laure Ligozat, Empreinte environnementale des modèles de langue, ETAL 2025, [https://github.com/run dimeco/ETAL2025\\_slides/blob/main/J05\\_P01\\_Anne-Laure\\_Ligozat.pdf](https://github.com/run dimeco/ETAL2025_slides/blob/main/J05_P01_Anne-Laure_Ligozat.pdf)

- Example 1: How much electricity does it take to train an LLM?

To compute the cost of training **BLOOM-176B**, Here is some useful information:

| Calculator parameter                   | Publicly reported data for BLOOM                      |
|----------------------------------------|-------------------------------------------------------|
| Runtime                                | 2837 hours, 41 minutes                                |
| CPU model                              | AMD EPYC 7543                                         |
| Number of cores used                   | 3072                                                  |
| GPU model                              | NVIDIA A100 SXM4 80GB                                 |
| Number of GPUs                         | 384 GPUs                                              |
| Training location                      | France, Jean Zay supercomputer                        |
| Memory (RAM) available during training | 512 GB per node * 48 training nodes: <b>24,576 GB</b> |

- Example 2: What is the environmental impact of using an LLM?

TODO: Discuss with a LLM, try to reimplement a simplified version of this tool.

## Tools you may need

```
# You can run this block to download the notebook with answers.  
from google.colab import files
```

```
files.download("ceipi-intro-programming/TD_en_answer.ipynb")
```

```
# Run this cell after editing tools.py to reload the module and make your changes available.  
import importlib  
import tools
```

```
# Reload tools.py so that newly added or updated functions become available.  
importlib.reload(tools)
```